Universidade Federal de Uberlândia - UFU
Faculdade de Computação - FACOM

Bacharelado em Sistema de Informação - Campus Monte Carmelo
Sistemas Distribuídos

Cidade Alerta

Rede de Emergência Urbana

Documentação detalhada do projeto distribuído

Nome: Pedro Henrique Vieira Pinto
Matrícula: 32221BSI012
Título do Projeto: Rede de Emergência Urbana

O documento foi escrito seguindo a seguinte organização:

1. Introdução
2. Arquitetura do Sistema
3. Implementação dos Conceitos de Sistema Distribuídos
    a. Processos e Threads
    b. Comunicação por Sockets
    c. Arquitetura baseada em mensagens
    d. Nomenclatura de processos
    e. Sincronização
        i. Relógios Lógicos
        ii. Exclusão Mútua
4. Integração do Middleware
5. Resultados e Testes
6. Conclusão
7. Instruções de Execução

- Links para vídeos de diferentes funcionamentos e testes realizados no projeto:
    - Funcionamento do projeto sem Middleware (teste em 1 máquina):
    - Funcionamento do projeto sem Middleware (teste em 4 máquinas):
    - Funcionamento do projeto com Middleware (teste em 4 máquinas, gravação de tela de 1 máquina):
    - Funcionamento do projeto com Middleware (teste com 4 máquinas, vídeo mostrando ambos por fora):

1. Introdução

Sistemas distribuídos estão presentes em diversas aplicações modernas, como sistemas de monitoramento, plataformas de serviços online, sistemas de emergência, GPS e etc. Esses sistemas são compostos por múltiplos processos executando em diferentes máquinas ou ambientes, que precisam se comunicar e coordenar suas ações para atingir um objetivo.

Este projeto tem como objetivo desenvolver um **micromundo distribuído** que simula um sistema de resposta a incidentes urbanos. Nesse ambiente, diferentes entidades cooperam para detectar, validar e responder a situações de emergência, como incêndios, acidentes ou ocorrências policiais. O sistema é composto por diversos componentes distribuídos, incluindo sensores de incidente, um centro de monitoramento, uma central de coordenação e unidades de atendimento especializadas, como ambulâncias, bombeiros, viaturas policiais e unidades de apoio logístico.

O funcionamento do sistema ocorre da seguinte forma: sensores simulam a detecção de incidentes e enviam essas informações para o sistema. O centro de monitoramento analisa os incidentes detectados e os valida antes de encaminhá-los para a central de coordenação. A central, por sua vez, é responsável por decidir quais unidades devem ser acionadas para cada tipo de incidente e enviar os comandos necessários para o atendimento da ocorrência. As unidades de atendimento então executam o atendimento do incidente e registram eventos operacionais durante o processo.

Ao longo do desenvolvimento do projeto, implementei diversos conceitos fundamentais de **Sistemas Distribuídos**, incluindo comunicação entre processos através de sockets, arquitetura baseada em mensagens, uso de um servidor de nomes para descoberta de serviços, sincronização distribuída utilizando relógios lógicos e controle de exclusão mútua para acesso a recursos compartilhados.

Além disso, na etapa final do projeto foi realizada a integração de um **middleware de mensageria distribuída**, o **Apache ActiveMQ Classic**, middleware sorteado a mim para ser implementado com o objetivo de melhorar a comunicação assíncrona entre os componentes do sistema. Com o uso do middleware, partes importantes da comunicação do sistema passaram a utilizar filas de mensagens, permitindo maior desacoplamento entre os processos e uma arquitetura mais próxima de sistemas distribuídos reais.

2. Arquitetura do Sistema

a. Visão Geral da Arquitetura

O sistema que desenvolvi neste projeto foi estruturado como um **micromundo distribuído** composto por múltiplos processos independentes que se comunicam entre si para detectar, validar e responder a incidentes simulados. Cada componente do sistema possui uma responsabilidade específica, permitindo que o sistema funcione de forma distribuída.

De forma geral, o fluxo de funcionamento do sistema ocorre em etapas. Inicialmente, sensores simulam a detecção de incidentes e enviam essas informações para o sistema. Essas informações são recebidas pelo Centro de Monitoramento, responsável por analisar e validar os incidentes antes de encaminhá-los para a Central de Coordenação. A Central então decide quais unidades de atendimento devem ser acionadas com base no tipo de incidente recebido.

Após essa decisão, comandos são enviados para as **Unidades de Atendimento**, que representam entidades responsáveis por atuar no incidente. Essas unidades executam uma sequência de ações operacionais, registrando eventos ao longo do atendimento e reportando seu estado ao sistema. Tudo é atualizado na central e registrado no banco.

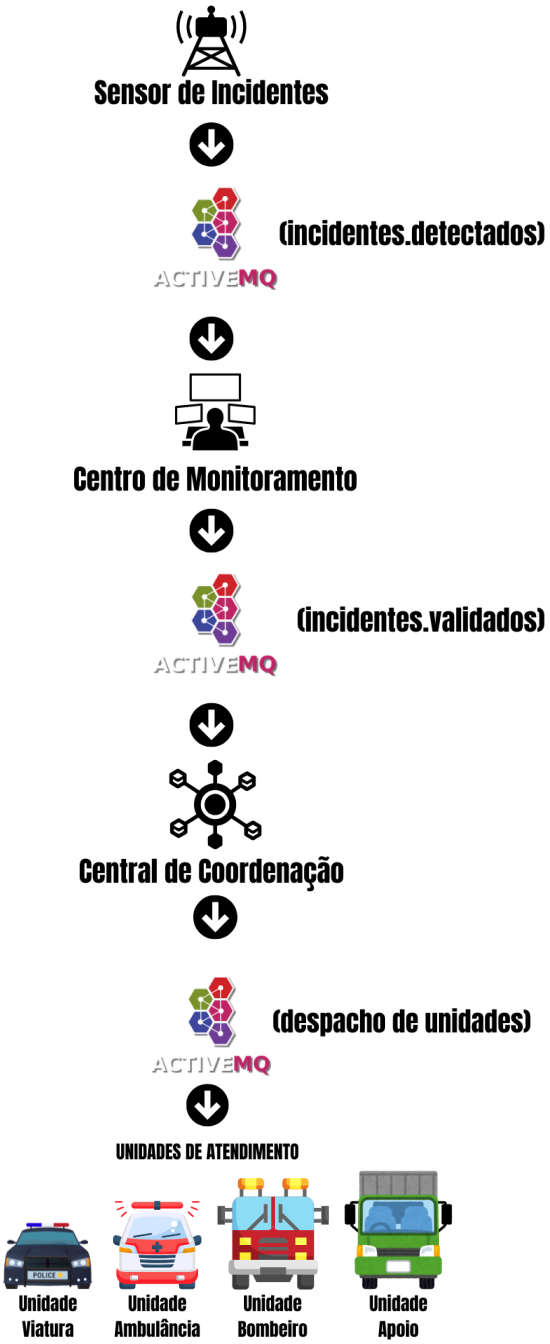
Para permitir a comunicação entre os componentes distribuídos, o sistema utiliza dois mecanismos principais:

- Comunicação direta via sockets TCP, utilizada para coordenação entre processos, controle de exclusão mútua e registro de eventos no banco de ocorrências.

- Comunicação baseada em mensageria através do middleware Apache ActiveMQ, utilizada para o envio assíncrono de incidentes e comandos entre os componentes do sistema.

Além disso, o sistema utiliza um **Servidor de Nomes**, responsável por permitir que os processos descubram dinamicamente a localização de outros serviços no sistema. Esse mecanismo evita a necessidade de configurações rígidas de endereço entre os componentes e facilita a inicialização do sistema distribuído.

A arquitetura geral do sistema pode ser representada pelo seguinte fluxo lógico:



Durante o atendimento das ocorrências, as unidades também enviam registros operacionais e informações de status para o sistema, permitindo o acompanhamento das ações realizadas no atendimento de cada incidente.

Organizando dessa forma, foi possível separar as responsabilidades que defini para cada componente/entidade e facilitou a evolução do sistema de acordo com as etapas propostas.

b. Componentes do Sistema

O sistema é composto por diferentes tipos de entidades distribuídas, cada uma responsável por uma parte específica do processamento de incidentes.

i. **Sensor de Incidentes**

O sensor de incidentes é responsável por simular (na realidade detectar/receber chamados e ligações) a detecção de eventos no ambiente monitorado. Esses eventos representam ocorrências como incêndios, acidentes de veículo, assaltos ou emergências médicas. Quando um incidente é detectado, o sensor cria uma mensagem contendo informações sobre o tipo de incidente, um identificador único e dados adicionais do evento. Essa mensagem é então publicada no middleware Apache ActiveMQ através da fila **incidentes.detectados**, permitindo que outros componentes do sistema recebam essa informação.

ii. **Centro de Monitoramento**

O Centro de Monitoramento atua como uma camada intermediária entre os sensores e a Central de Coordenação. Seu papel principal é analisar os incidentes detectados e validar as informações recebidas antes de encaminhá-las para o sistema de coordenação.

Ao receber um incidente do middleware, o Centro de Monitoramento pode realizar verificações ou enriquecimento das informações antes de publicar o incidente validado na fila **incidentes.validados**. Essa etapa representa o processo de triagem ou análise inicial de ocorrências

iii. **Central de Coordenação**

A Central de Coordenação é o componente responsável por gerenciar o fluxo principal do sistema. Ela recebe os incidentes validados e decide quais unidades devem ser acionadas para atender cada ocorrência.

A Central mantém uma **fila de incidentes pendentes**, permitindo que múltiplos incidentes sejam tratados de forma ordenada. Para cada incidente processado, a Central consulta um mapeamento de tipos de incidente para unidades de atendimento e publica comandos de despacho nas filas apropriadas do middleware.

Além disso, a Central também é responsável por coordenar o acesso ao **banco de ocorrências**, garantindo que múltiplas unidades não registrem eventos simultaneamente sem controle de sincronização.

iv. **Unidades de Atendimento**

As unidades de atendimento representam os agentes responsáveis por atuar diretamente nos incidentes. Cada unidade possui um comportamento específico de acordo com sua função no sistema.

1. **Ambulância:** responsável por atendimentos médicos emergenciais.

2. **Bombeiros:** responsáveis por ocorrências como incêndios e desabamentos.
3. **Viatura Policial:** responsável por ocorrências policiais, como assaltos ou apoio em acidentes.
4. **Unidade de Apoio Logístico:** responsável por suporte operacional em incidentes complexos.

Cada unidade recebe comandos de atendimento através de uma fila específica do middleware, executa uma sequência de ações simuladas e registra eventos relacionados ao atendimento do incidente.

v. **Servidor de Nomes**

O servidor de Nomes é responsável por fornecer um mecanismo de descoberta de serviços dentro do sistema distribuído. Ele mantém um registro das entidades ativas no sistema e permite que outros processos consultem o endereço e a porta de comunicação de cada serviço.
Esse mecanismo permite que os componentes descubram a localização da Central de Coordenação sem depender de configurações fixas no código.

c. Comunicação entre Componentes

A comunicação entre os componentes do sistema ocorre utilizando dois modelos distintos.
O primeiro modelo é a comunicação direta via sockets TCP, utilizada principalmente para coordenação entre processos e troca de mensagens críticas, como controle de exclusão mútua e registro de eventos no banco de ocorrências.
O segundo modelo é baseado em mensageria assíncrona através do middleware Apache ActiveMQ, onde mensagens são enviadas para filas específicas e consumidas pelos componentes interessados. Esse modelo permite desacoplar os produtores e consumidores de mensagens, facilitando a escalabilidade e organização do sistema.

d. Filas e Mensageria do Middleware

Durante a execução do sistema, o middleware Apache ActiveMQ utiliza diferentes filas para organizar a comunicação entre os componentes.
As principais filas utilizadas no projeto são:

- incidentes.detectados: utilizada pelos sensores para publicar novos incidentes detectados.
- incidentes.validados: utilizada pelo Centro de Monitoramento para publicar incidentes analisados e validados.
- despacho.ambulancia: utilizada pela Central para enviar comandos para unidades de ambulância.
- despacho.bombeiros: utilizada para acionar equipes de bombeiros.

- despacho.viatura: utilizada para despachar viaturas policiais.
- despacho.apoio: utilizada para enviar comandos para unidades de apoio logístico.
- status.unidades: utilizada pelas unidades para reportar eventos operacionais e estado do atendimento.

3. Implementação dos Conceitos de Sistema Distribuídos

Durante o desenvolvimento do projeto, diversos conceitos fundamentais de Sistemas Distribuídos foram aplicados na construção da arquitetura e na comunicação entre os componentes do sistema, ou seja, a implementação de cada etapa proposta ao longo da disciplina.

Esta seção descreve como cada um desses conceitos foi implementado no projeto e como eles contribuem para o funcionamento correto do sistema distribuído.

a. Processos e Threads

Estruturei o projeto como um conjunto de processos distribuídos independentes, onde cada entidade do micromundo executa como um programa separado(Separei por arquivos de código).
Cada uma das 8 entidades que implementei executam como um processo independente e possui responsabilidades específicas dentro do sistema.

Além da divisão em processos, algumas entidades utilizam threads internas para executar tarefas concorrentes. Um exemplo importante ocorre na Central de Coordenação, que utiliza uma thread dedicada para processar continuamente a fila de incidentes recebidos. Enquanto essa thread processa os incidentes, a thread principal permanece responsável por aceitar conexões e receber mensagens de outras entidades.

De forma semelhante, as unidades de atendimento utilizam threads para processar chamados recebidos enquanto continuam escutando novas mensagens vindas da Central de Coordenação ou do middleware. Essa estrutura permite que diferentes partes do sistema operem simultaneamente, simulando um comportamento mais próximo de sistemas distribuídos reais.

b. Comunicação por Sockets

Parte da comunicação entre os componentes do sistema foi implementada utilizando sockets de rede, que permitem a troca direta de mensagens entre processos distribuídos.
O sistema está utilizando dois tipos principais de comunicação por sockets:

- **Comunicação TCP**

Sockets TCP são utilizados para comunicação confiável entre os processos, principalmente em situações que exigem coordenação direta entre as entidades.
Esse tipo de comunicação é utilizado, por exemplo, na interação entre as unidades de atendimento e a Central de Coordenação.
Através dessa comunicação, as unidades enviam eventos operacionais, solicitam acesso á seção crítica para registrar ocorrências e recebem autorizações da Central para realizar determinadas operações.

- **Comunicação UDP**
  Sockets UDP são utilizados pelo servidor de nomes, responsável por registrar e consultar a localização das entidades do sistema. Como esse tipo de comunicação envolve mensagens curtas e simples, o uso de UDP permitiu uma comunicação mais leve e adequada para esse tipo de serviço.
  Quando uma entidade inicia sua execução, ela registra seu nome e porta de comunicação no servidor de nomes. Outras entidades podem então consultar esse servidor para descobrir o endereço de um serviço específico.

c. Arquitetura baseada em mensagens

A comunicação entre os componentes do sistema é baseada em troca de mensagens estruturadas, permitindo que diferentes entidades troquem informações de forma padronizada.
As mensagens do sistema são representadas em formato JSON, contendo campos que identificam o tipo de mensagem, a entidade de origem, o conteúdo da mensagem e informações de sincronização, como os relógios lógicos.

- EVENTO: registro de eventos operacionais de uma unidade
- INCIDENTE_MONITORADO: incidente validado pelo Centro de Monitoramento
- COMANDO: comando enviado pela Central de Coordenação para uma unidade
- REQUEST_SC: solicitação de acesso à seção crítica
- GRANT_SC: autorização para acessar a seção crítica
- RELEASE_SC: liberação da seção crítica
- LIDER_ELEITO: mensagem de confirmação de liderança

d. Nomenclatura de processos

Para facilitar a identificação das entidades dentro do sistema distribuído, foi adotado um esquema de nomenclatura padronizada de processos. Cada entidade possui um identificador único que representa sua função no sistema.
Alguns exemplos de nomes que utilizei no projeto:

- CENTRAL_COORDENAÇÃO

- AMBULANCIA_01
- BOMBEIROS_01
- VIATURA_POLICIAL_01
- UNIDADE_APOIO_01

Esses nomes são utilizados nas mensagens trocadas entre os processos e também no mecanismo de registro do Servidor de Nomes.

Quando uma entidade inicia sua execução, ela envia uma mensagem de registro ao servidor de nomes contendo seu identificador e sua porta de comunicação. Dessa forma, outras entidades podem consultar o servidor e localizar dinamicamente o endereço de um determinado processo.

e. Sincronização

Sabemos que em sistemas distribuídos, os processos podem tentar acessar recursos compartilhados simultaneamente, o que causa conflitos. Então, para evitar esse tipo de problema, implementei os mecanismos de sincronização no projeto. Também parte das etapas propostas para o projeto.

i. Relógios Lógicos

Pra manter uma ordenação consistente dos eventos no sistema, implementei relógios de Lamport e relógios vetoriais.
Cada uma das entidades do sistema mantém um contador lógico que é incrementado sempre que ocorre um evento local ou quando uma mensagem é enviada. Quando uma entidade recebe uma mensagem de outra entidade, ela atualiza seu relógio lógico com base no valor recebido, garantido que a ordem dos eventos seja preservada.
Além dos relógios de Lamport, cada entidade também mantém um vetor de relógios, onde cada posição representa o estado lógico de um processo específico do sistema.
Inclui esses relógios nas mensagens trocadas entre os processos e também são registrados no banco de ocorrências, permitindo rastrear a ordem lógica que os eventos ocorreram no sistema, durante o seu funcionamento.

ii. Exclusão Mútua

O sistema possui um recurso compartilhado que chamei de Banco de Ocorrências, responsável por armazenar os registros de eventos relacionados ao atendimento de incidentes. Como várias unidades podem tentar registrar ao mesmo tempo, coloquei o mecanismo de exclusão mútua distribuída para controlar o acesso a esse recurso.
O controle de acesso é realizado pela central, que mantém uma fila de solicitações de acesso enviadas pelas unidades.
Quando uma unidade precisa registrar um evento no banco de ocorrências, ele envia uma mensagem do tipo REQUEST_SC para a

central. A central adiciona essa solicitação a uma fila ordenada pelo relógio lógico de Lamport.
Quando a seção fica disponível, a Central envia uma mensagem GRANT_SC autorizando a unidade a acessar o banco de ocorrências.
Após concluir o registro, a unidade envia uma mensagem RELEASE_SC, indicando que o recurso pode ser liberado para o próximo processo na fila.
Sendo assim, o foco desse mecanismo foi controlar o acesso ao banco para evitar os conflitos de acesso.

4. Integração do Middleware

Na etapa final do projeto, realizei a integração do Middleware de mensageria com o objetivo de tornar a comunicação entre os componentes do sistema mais desacoplada e mais próxima da arquitetura utilizada em sistemas reais.

O Middleware que utilizei foi o **Apache ActiveMQ Classic**, um sistema de mensageria baseada no padrão MOM (Mensageria Orientada a Middleware). Esse tipo de middleware permite que aplicações distribuídas se comuniquem através do envio e recebimento de mensagens organizadas em filas ou tópicos, sem que os processos precisem se conectar diretamente entre si.

a. Apache ActiveMQ Classic

O Apache ActiveMQ Classic é um broker de mensagens amplamente utilizado em arquiteturas distribuídas. Ele atua como um intermediário responsável por receber mensagens de um produtor e entregá-las a um ou mais consumidores interessados.
No contexto deste projeto, o ActiveMQ foi utilizado como um broker central de comunicação, permitindo que diferentes componentes do sistema troquem informações através de filas de mensagens.
Uma das principais características desse tipo de arquitetura é que o produtor de uma mensagem não precisa conhecer diretamente o consumidor da informação. Ele apenas envia a mensagem para uma fila específica, e o broker se encarrega de entregar essa mensagem para os consumidores inscritos naquela fila.

b. Integração do Middleware no Sistema

A integração do middleware foi realizada principalmente nas etapas de detecção de incidentes, validação de eventos e despacho de unidades de atendimento.
Antes da integração do middleware, a comunicação entre os componentes ocorria principalmente através de conexões diretas entre processos utilizando sockets. Com a entrada do ActiveMQ, parte dessa comunicação passou a ser realizada através de filas de mensagens, permitindo uma comunicação mais flexível.

O fluxo da comunicação utilizando o middleware ocorre da seguinte forma:

i. **Sensor de Incidentes:** Detecta um evento e publica uma mensagem na fila incidentes.detectados.
ii. **Centro de Monitoramento:** Consome mensagens dessa fila, analisa o incidente e publica um incidente validado na fila incidentes.validados.
iii. **Central de Coordenação:** Consome os incidentes validados e decide quais unidades devem ser acionadas para resolver o problema.
iv. **Central:** Publica comandos de despacho em filas específicas para cada tipo de unidade de atendimento.
v. **Unidades de Atendimento:** Consomem as mensagens dessas filas e iniciam o atendimento do incidente.
vi. **Por fim:** Durante o atendimento, as unidades publicam mensagens de status na fila status.unidade, permitindo acompanhar o progresso do atendimento.

c. Problemas resolvidos com o Middleware

A integração do middleware trouxe algumas melhorias importantes em relação à arquitetura anterior baseada apenas em comunicação direta entre processos. Uma das principais melhorias foi o desacoplamento entre produtores e consumidores de mensagem. No modelo antigo, antes da chegada do middleware, o envio de incidentes e comandos exigia que os processos estivessem diretamente ligados entre si. Com o uso do ActiveMQ, os componentes apenas publicam ou consomem mensagens em filas específicas, sem precisar coinhecer as demais entidades.

Outra coisa que melhorou foi a comunicação assíncrona entre os componentes. Com o uso de filas de mensagens, os processos podem continuar executando suas tarefas mesmo que o consumidor da mensagem ainda não tenha processado o evento.

Além disso, o middleware também facilitou muito na organização da comunicação do sistema, pois cada mensagem passou a utilizar sua fila específica.

d. Consistência, Replicação, Tolerância a Falhas e Segurança (Com base em pesquisas que realizei)

Sobre a consistência, o ActiveMQ garante que as mensagens enviadas para uma fila sejam entregues aos consumidores de forma ordenada dentro daquela fila, preservando a sequência de eventos publicada pelas produtores.

Em relação à replicação e tolerância a falhas, o ActiveMQ oferece recursos que permitem replicar mensagens e manter cópias dos dados em diferentes nós do sistema. Em configurações mais avançadas, isso possibilita que o sistema continue operando mesmo em caso de falha de um dos componentes do broker.

Quanto à segurança, o middleware também oferece mecanismos de autenticação, controle de acesso e comunicação segura entre clientes e o broker. Esses recursos permitem restringir quais aplicações podem publicar ou consumir mensagens em determinadas filas.

e. Desafios na Integração do Middleware

Um dos principais desafios foi adaptar a arquitetura antiga que era baseada em comunicação direta entre processos, para utilizar a comunicação baseada em filas por mensagens.
Precisei identificar quais partes da comunicação do sistema poderiam ser substituídas pelo uso do middleware e adaptar os componentes para publicar e consumir mensagens através do ActiveMQ.

5. Resultados e Testes

Após a implementação de todos os componentes do sistema distribuído e a integração do middleware Apache ActiveMQ, foram realizados testes para verificar o funcionamento correto da arquitetura proposta.
Durante os testes, cada entidade do sistema foi executada em um terminal separado, simulando um ambiente distribuído no qual diferentes processos operam simultaneamente e se comunicam através da rede e do middleware.

Considerando que todos as entidades foram iniciadas e todas ja estão prontas para funcionamento.

Início o teste mostrando a situação inicial da seção de filas na interface do Apache ActiveMQ Classic.

Queues:

Name ↑	Number Of Pending Messages	Number Of Consumers	Messages Enqueued	Messages Dequeued	Views	Operations
despacho.ambulancia	0	1	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
despacho.apoio	0	1	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
despacho.bombeiros	0	1	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
despacho.viatura	0	1	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
incidentes.detectados	0	1	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
incidentes.validados	0	1	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
status.unidades	4	0	4	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
teste	0	0	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause

As mensagens pendentes referem-se ao status atual das 4 entidades: Ambulância, Bombeiros, Viaturas e Unidades de apoio.

O primeiro passo foi ir até o Sensor de Incidentes e provocar um incidente, o escolhido do teste foi o incêndio.

```
[SENSOR_INCIDENTE] Iniciando sensor com middleware ActiveMQ...
[SENSOR_INCIDENTE] Fila de publicação: /queue/incidentes.detectados
[SENSOR_INCIDENTE] Conectado ao ActiveMQ em localhost:61613
[SENSOR_INCIDENTE] Sensor ativo e pronto para registrar incidentes.


========== MENU DE INCIDENTES ==========
1 - Incêndio
2 - Acidente de veículo
3 - Assalto
4 - Desabamento
5 - Emergência médica
0 - Encerrar sensor
========================================

Selecione o incidente desejado: 1
[SENSOR_INCIDENTE] Incidente publicado com sucesso -> ID=INC-1 | TIPO=INCENDIO | LOCAL=(23,92) | Lamport=4
```

Após escolher a opção 1, fui ao centro de monitoramento verificar a validação do incidente.

```
[MONITORAMENTO] Registrado no Servidor de Nomes com confirmação. | Lamport=8
[MONITORAMENTO] Localizando Central de Coordenação...
[MONITORAMENTO] Conectado à Central em 127.0.0.1:5000
[MONITORAMENTO] Status: monitoramento ativo, aguardando incidentes do ActiveMQ.
[ELEIÇÃO] Solicitação de eleição enviada à Central. | Lamport=15
[MONITORAMENTO] Conectado ao ActiveMQ em localhost:61613
[MONITORAMENTO] Aguardando incidentes do broker em /queue/incidentes.detectados...
[MONITORAMENTO] Incidente recebido do broker | Lamport=16
[MONITORAMENTO] Incidente validado -> ID=INC-1 | TIPO=INCENDIO | GRAVIDADE=ALTA | LOCAL=(23,92)
[ENVIO] Incidente monitorado publicado em /queue/incidentes.validados | Lamport=19
```

O centro de monitoramento foi iniciado, se conectou ao ActiveMQ e começou aguardar por incidentes.
Recebeu uma mensagem do broker (Lamport 16), validou registrando como INC-1 e logo publicou na fila de incidentes validados. (Lamport 19).
O próximo passo foi verificar a Central de Monitoramento.

```
==============================================================================
[INCIDENTE RECEBIDO VIA ACTIVEMQ] ID=INC-1 | TIPO=INCENDIO | Origem=CENTRO_MONITORAMENTO | Lamport=29
==============================================================================

==============================================================================
[CENTRAL] Incidente enfileirado -> ID=INC-1 | TIPO=INCENDIO
[CENTRAL] Total na fila: 1
==============================================================================

==============================================================================
[CENTRAL] PROCESSANDO INCIDENTE -> ID=INC-1 | TIPO=INCENDIO | Lamport=30
==============================================================================

------------------------------------------------------------------------------
[BANCO] Registro salvo | Origem=CENTRAL_COORDENACAO | Incidente=INC-1 | Evento=Incidente INC-1 recebido e encaminhado para despacho. | Lamport=31

==============================================================================
[CENTRAL] DESPACHANDO UNIDADES VIA ACTIVEMQ -> INC-1 (INCENDIO)
==============================================================================

------------------------------------------------------------------------------
[DESPACHO] Comando publicado para BOMBEIROS_01 em /queue/despacho.bombeiros | Incidente=INC-1

------------------------------------------------------------------------------
[DESPACHO] Comando publicado para UNIDADE_APOIO_01 em /queue/despacho.apoio | Incidente=INC-1

------------------------------------------------------------------------------
[DESPACHO] Comando publicado para VIATURA_POLICIAL_01 em /queue/despacho.viatura | Incidente=INC-1

------------------------------------------------------------------------------
[DESPACHO] Comando publicado para AMBULANCIA_01 em /queue/despacho.ambulancia | Incidente=INC-1
```

A Central recebeu o incidente com o ID, tipo, origem e lamport de 29.
Enfileirou o incidente, logo depois processou pois não tinha outros na fila.
Ao Lamport 31, o incidente foi registrado no banco de ocorrências.
Após isso, a central iniciou o processo de despache das unidades necessárias para gerir a situação.
Foi publicado um comando de despacho na fila de cada uma das unidades.

Após isso, as unidades começaram a informar os deslocamentos para o local do incidente e também solicitaram acesso ao banco para registrar os eventos.

```
--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: AMBULANCIA_01

--------------------------------------------------------------------------------
[EVENTO RECEBIDO de AMBULANCIA_01] Deslocamento iniciado para o incidente INC-1. | Lamport=38

--------------------------------------------------------------------------------
[BANCO] Registro salvo | Origem=AMBULANCIA_01 | Incidente=INC-1 | Evento=Deslocamento iniciado para o incidente INC-1. | Lamport=39

--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: AMBULANCIA_01

--------------------------------------------------------------------------------
[REQUEST_SC] AMBULANCIA_01 solicitou acesso ao banco | Lamport=40
[FILA SC] Estado atual: ['AMBULANCIA_01']

--------------------------------------------------------------------------------
[GRANT_SC] Acesso concedido para AMBULANCIA_01

--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: VIATURA_POLICIAL_01

--------------------------------------------------------------------------------
[EVENTO RECEBIDO de VIATURA_POLICIAL_01] Deslocamento da viatura para apoio operacional no incidente INC-1. | Lamport=42

--------------------------------------------------------------------------------
[BANCO] Registro salvo | Origem=VIATURA_POLICIAL_01 | Incidente=INC-1 | Evento=Deslocamento da viatura para apoio operacional no incidente INC-1. | Lamport=43

--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: VIATURA_POLICIAL_01

--------------------------------------------------------------------------------
[REQUEST_SC] VIATURA_POLICIAL_01 solicitou acesso ao banco | Lamport=44
[FILA SC] Estado atual: ['VIATURA_POLICIAL_01']
```

Abaixo visualizando como exemplo o terminal da Ambulância durante o movimento provocado pelo incidente.

```
[AMBULÂNCIA] Localizando Central de Coordenação...
[AMBULÂNCIA] Conectada ao ActiveMQ em localhost:61613
[AMBULÂNCIA] Conectada à Central em 127.0.0.1:5000
[AMBULÂNCIA] Status: unidade médica disponível e aguardando chamados.
[AMBULÂNCIA] STATUS publicado no broker | Incidente=None | Unidade médica online e aguardando chamados.
[AMBULÂNCIA] Aguardando comandos do broker em /queue/despacho.ambulancia...
[AMBULÂNCIA] COMANDO recebido via ActiveMQ -> Atender incidente INC-1 (INCENDIO)
[AMBULÂNCIA] Novo chamado recebido -> Incidente=INC-1 | Tipo=INCENDIO | Local=(23,92)
[AMBULÂNCIA] STATUS publicado no broker | Incidente=INC-1 | Deslocamento iniciado para o incidente INC-1.
[AMBULÂNCIA] EVENTO enviado | Incidente=INC-1 | Deslocamento iniciado para o incidente INC-1. | Lamport=38
[SC] REQUEST_SC enviado | Incidente=INC-1 | Lamport=39
[SC] GRANT_SC recebido! Acesso liberado para incidente INC-1.
[RELÓGIOS] Lamport=42 | Vetor=[21, 26, 10, 4, 9, 3, 3, 3] | Líder=CENTRAL_COORDENACAO
[SC] Registrando ocorrência | Incidente=INC-1 | Evento=Deslocamento iniciado para o incidente INC-1.
[SC] RELEASE_SC enviado | Incidente=INC-1 | Lamport=44
[AMBULÂNCIA] STATUS publicado no broker | Incidente=INC-1 | Chegada ao local do incidente INC-1.
[AMBULÂNCIA] EVENTO enviado | Incidente=INC-1 | Chegada ao local do incidente INC-1. | Lamport=46
[SC] REQUEST_SC enviado | Incidente=INC-1 | Lamport=47
[SC] GRANT_SC recebido! Acesso liberado para incidente INC-1.
[RELÓGIOS] Lamport=74 | Vetor=[21, 54, 10, 4, 15, 11, 11, 11] | Líder=CENTRAL_COORDENACAO
[SC] Registrando ocorrência | Incidente=INC-1 | Evento=Chegada ao local do incidente INC-1.
[SC] RELEASE_SC enviado | Incidente=INC-1 | Lamport=76
[AMBULÂNCIA] STATUS publicado no broker | Incidente=INC-1 | Atendimento médico iniciado no incidente INC-1.
[AMBULÂNCIA] EVENTO enviado | Incidente=INC-1 | Atendimento médico iniciado no incidente INC-1. | Lamport=78
```

A ambulância se conecta a central, atualiza um status indicando que está disponível para chamados, pública o status no broker.
Após isso, recebe o chamado, informa o deslocamento e logo solicita acesso ao banco de ocorrências para registrar o evento. Depois, publica status informando que chegou no local, registra novamente no banco de ocorrências, informa que o atendimento médico iniciou no incidente INC-1 e se repete o registro no banco.

Após isso, as outras entidades envolvidas no incidente repetem o mesmo processo e no final, todas informam que estão livres aguardando novos chamados.

Queues:

Name ↑	Number Of Pending Messages	Number Of Consumers	Messages Enqueued	Messages Dequeued	Views	Operations
despacho.ambulancia	0	1	1	1	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
despacho.apoio	0	1	1	1	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
despacho.bombeiros	0	1	1	1	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
despacho.viatura	0	1	1	1	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
incidentes.detectados	0	1	1	1	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
incidentes.validados	0	1	1	1	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
status.unidades	27	0	27	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause
teste	0	0	0	0	Browse Active Consumers Active Producers atom rss	Send To Purge Delete Pause

Ao fim, às filas de cada entidade estão zeradas pois não incidente para ser socorrido, exceto o status.unidades que está guardando todas as atualizações feitas durante o atendimento.
As mensagens enfileiradas e desenfileiradas guardam o valor de 1 pois atendemos 1 incidente e após ele não foram publicados comando de despache de cada uma das entidades.

```
--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: VIATURA_POLICIAL_01

--------------------------------------------------------------------------------
[RELEASE_SC] VIATURA_POLICIAL_01 liberou a seção crítica | Lamport=190

--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: AMBULANCIA_01

--------------------------------------------------------------------------------
[EVENTO RECEBIDO de AMBULANCIA_01] Unidade médica disponível novamente após atendimento do incidente INC-1. | Lamport=191

--------------------------------------------------------------------------------
[BANCO] Registro salvo | Origem=AMBULANCIA_01 | Incidente=INC-1 | Evento=Unidade médica disponível novamente após atendimento do incidente INC-1. | Lamport=192

--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: BOMBEIROS_01

--------------------------------------------------------------------------------
[EVENTO RECEBIDO de BOMBEIROS_01] Equipe de bombeiros disponível novamente após atendimento do incidente INC-1. | Lamport=193

--------------------------------------------------------------------------------
[BANCO] Registro salvo | Origem=BOMBEIROS_01 | Incidente=INC-1 | Evento=Equipe de bombeiros disponível novamente após atendimento do incidente INC-1. | Lamport=
194

--------------------------------------------------------------------------------
[CENTRAL] Conexão associada à entidade: VIATURA_POLICIAL_01

--------------------------------------------------------------------------------
[EVENTO RECEBIDO de VIATURA_POLICIAL_01] Viatura disponível novamente após atendimento do incidente INC-1. | Lamport=195

--------------------------------------------------------------------------------
[BANCO] Registro salvo | Origem=VIATURA_POLICIAL_01 | Incidente=INC-1 | Evento=Viatura disponível novamente após atendimento do incidente INC-1. | Lamport=196
```

Por fim, as entidades informam a central que estão disponíveis novamente após atendimento ao INC-1.

6. Conclusão

O desenvolvimento deste projeto permitiu aplicar, de forma prática, diversos conceitos fundamentais da área de Sistemas Distribuídos. A construção do micromundo distribuído demonstrou como diferentes processos independentes podem cooperar para resolver um problema comum, neste caso a detecção, validação e atendimento de incidentes simulados em um ambiente urbano.

Ao longo do projeto, foram implementados mecanismos importantes como comunicação entre processos utilizando sockets, descoberta de serviços através de um servidor de nomes, arquitetura baseada em troca de mensagens, sincronização distribuída com relógios lógicos e controle de exclusão mútua para acesso a recursos compartilhados. Esses elementos permitiram explorar desafios típicos de sistemas distribuídos, como coordenação entre processos, ordenação de eventos e concorrência no acesso a dados.

De forma geral, o projeto cumpriu seu objetivo de demonstrar na prática o funcionamento de um sistema distribuído, permitindo aplicar conceitos teóricos em um cenário simulado e compreender melhor os desafios envolvidos na construção desse tipo de sistema.

7. Instruções de Execução

Esta seção descreve, em detalhes, como preparar o ambiente, instalar as dependências, iniciar o middleware e executar o projeto.

7.1 Requisitos do ambiente

Para executar o projeto, é necessário possuir:

- Python 3.x
- biblioteca Python stomp.py
- Apache ActiveMQ Classic
- múltiplos terminais abertos para execução simultânea dos processos

O Apache ActiveMQ Classic possui distribuições oficiais para Windows em formato .zip. Na página oficial de download, as linhas estáveis suportadas incluem, por exemplo, 6.2.1 e 5.19.2; a linha 6.2.1 requer Java 17+, enquanto a linha 5.19.2 requer Java 11+.

Download Apache ActiveMQ

7.2 Descompactação dos arquivos do projeto

O projeto estará em formato compactado, descompacte e verifique se a estrutura é a esperada:

➔ Central_coordenação

- ➔ centro_monitoramento
- ➔ cofig_rede
- ➔ sensor_incidente
- ➔ servidor_nomes
- ➔ unidade_ambulancia
- ➔ unidade_apoio
- ➔ unidade_bombeiros
- ➔ unidade_viatura

7.3 Instalação do Python

Caso o Python ainda não esteja instalado, o primeiro passo é realizar sua instalação.

1. Acessar o site oficial do Python.
2. Baixar a versão mais recente compatível com Python 3.
3. Durante a instalação, marcar a opção **Add Python to PATH**.
4. Concluir a instalação.

7.4 Instalação da biblioteca stomp.py

A comunicação com o ActiveMQ foi feita utilizando o protocolo STOMP, que é suportado oficialmente pelo ActiveMQ Classic e é apropriado para clientes em Python**.**

Para instalar a biblioteca necessária no Python, executar no terminal:

```
python -m pip install stomp.py
```

Após a instalação, é possível confirmar que a biblioteca foi instalada corretamente com:

```
python -m pip show stomp.py
```

7.5 Inicialização do middleware

Depois de instalar o Java e descompactar o ActiveMQ, o middleware pode ser iniciado.

1. Abra um terminal.
2. Navegue até a pasta bin do ActiveMQ.
3. Execute o comando de inicialização: `activemq.bat start`

7.6 Verificação do painel administrativo

Após iniciar o ActiveMQ, o painel administrativo pode ser acessado no navegador pelo endereço: http://localhost:8161

usuário: admin

senha: admin

7.11 Ordem de execução dos componentes

Após iniciar o middleware, os demais componentes devem ser executados em terminais separados, obedecendo a seguinte ordem:

1. Servidor de Nomes
2. Central de Coordenação
3. Centro de Monitoramento

4. Unidade de Ambulância
5. Unidade de Apoio
6. Unidade de Bombeiros
7. Unidade de Viatura
8. Sensor de Incidente

7.12 Execução de um teste funcional

Após todos os componentes estarem ativos, o teste do sistema pode ser realizado pelo terminal do Sensor de Incidentes.

O sensor apresentará um menu de incidentes. Ao selecionar uma opção, o fluxo esperado é:

1. O sensor publica o incidente na fila incidentes.detectados.
2. O Centro de Monitoramento consome o incidente, valida a mensagem e publica em incidentes.validados.
3. A Central consome o incidente validado e despacha as unidades necessárias.
4. As unidades consomem os comandos de despacho em suas filas específicas.
5. Durante o atendimento, as unidades publicam status em status.unidades e também registram eventos no banco de ocorrências através do mecanismo de exclusão mútua.

Durante a execução, o painel do ActiveMQ deve exibir filas como:
incidentes.detectados
incidentes.validados
despacho.ambulancia
despacho.bombeiros
despacho.viatura
despacho.apoio
status.unidades

7.13 Encerramento do sistema

Para encerrar o sistema:

1. interromper manualmente os processos Python nos terminais;
2. parar o ActiveMQ com o comando: activemq.bat stop